

CloudStore

Design and Simulation of a Distributed File Storage System

System Design - Final Examination Project Report

Submitted by: Yash Khapre

Roll no:150096724027

B.Tech, Computer Science

Course: System Design

1. Problem Statement

CloudStore is a cloud-based file storage platform (similar to Google Drive) that lets users upload, store, access, and share files from any device. As CloudStore scales to millions of users, it must address:

1.1 Functional Requirements

- Users can upload, edit, delete, and share files.
- Users can share files with other users with defined access levels (read/write).
- Files are automatically synchronized across a user's devices and with users they are shared with.
- On mobile devices, file synchronization can be on-demand to save bandwidth.
- The system supports versioning, so previous versions of a file can be retained/restored.
- The system must handle conflicts when the same file is updated from multiple devices.

1.2 Non-Functional Requirements

- The system must be highly available and fault-tolerant, with no single point of failure.
- The design should scale horizontally to support millions of users and growing storage volume.
- Storage space and network bandwidth usage should be minimized (e.g. via chunking and deduplication).
- Uploads/downloads should be fast, even for large files.
- Data must be durable - no data loss even if individual storage nodes fail.
- Access control must be secure - only authorized users can view or modify a file.

2. Proposed Solution

CloudStore is designed as a set of independent services (microservices), each responsible for one concern and each with its own database. Key decisions:

- Chunked uploads/downloads - large files are split into fixed-size chunks, enabling parallel transfer, resumable uploads, and deduplication.
- Metadata stored separately from file content - small relational metadata vs. large unstructured blob data.

- Replication across storage nodes - every chunk is stored on multiple nodes for durability and fault tolerance.
- Gateway + load balancer layer - allows horizontal scaling of services independently.
- Notification service with offline backlog - keeps multiple devices in sync in real time.
- Database-per-service - Metadata, Users & Devices, and Billing each have their own database to avoid a single bottleneck.

3. System Architecture

Client apps (web, desktop, mobile) connect through a Load Balancer and API Gateway, which routes requests to the Metadata Service, Chunk Service, Block Service, Notification Service, and Users & Devices Service.

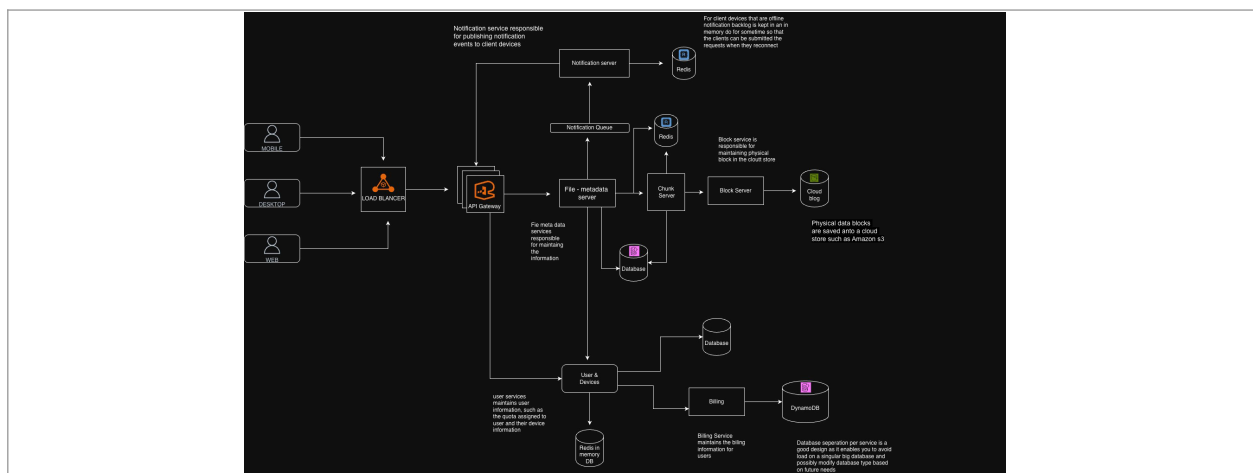


Fig 1: CloudStore High-Level Architecture Diagram

3.1 Upload Flow

- Client -> Load Balancer -> API Gateway (authenticates request)
- Gateway -> Metadata Service (creates/updates file record)
- Chunk Service splits file into chunks, hashes each, skips chunks already stored (dedup)
- Block Service writes new chunks to storage nodes with replication
- Notification Service informs other devices of the change

3.2 Download Flow

- Gateway checks access permission via Metadata Service
- Chunk Service returns ordered list of chunk hashes for the requested version
- Each chunk is read from any available replica and reassembled into the file

4. Module Description

Module	Responsibility
--------	----------------

API Gateway	Single entry point; authenticates and routes requests; holds long-lived connections for sync notifications.
Load Balancer	Distributes traffic across multiple Gateway instances.
Metadata Service	Stores file names, paths, owners, versions, and sharing permissions.
Chunk Service	Splits files into chunks, hashes for dedup, tracks chunk order per version.
Block Service / Storage Nodes	Physically stores chunks, replicated across multiple nodes.
Notification Service	Pushes sync events to devices; queues a backlog for offline devices.
Users & Devices Service	Manages user accounts, storage quotas, and registered devices.
Billing Service	Tracks subscription/billing information per user.

5. Database Design

Each service owns its own database (SQLite in this simulation; PostgreSQL/NoSQL in production).

5.1 Metadata DB - files

Column	Type	Description
file_id	TEXT (PK)	Unique file identifier (UUID).
file_name	TEXT	Display name of the file.
owner_id	TEXT	Owner's user ID.
folder_path	TEXT	Logical folder location.
size_bytes	INTEGER	Size of current version.
latest_version	INTEGER	Most recent version number.
created_at / updated_at	REAL	Timestamps.

5.2 Metadata DB - permissions

Column	Type	Description
--------	------	-------------

file_id	TEXT (PK, FK)	References files.file_id.
user_id	TEXT (PK)	User the permission applies to.
access_level	TEXT	'owner', 'read', or 'write'.

5.3 Chunk DB - file_chunks

Column	Type	Description
file_id	TEXT (PK, FK)	References files.file_id.
version	INTEGER (PK)	Version this chunk belongs to.
chunk_index	INTEGER (PK)	Order of chunk within the file.
chunk_hash	TEXT	SHA-256 hash; used as storage address and for dedup.
chunk_size	INTEGER	Size of chunk in bytes.

5.4 Users & Devices DB

Table	Key Columns	Description
users	user_id (PK)	name, quota_bytes, used_bytes - tracks per-user storage quota.
devices	device_id (PK)	user_id (FK), device_name, last_sync - registered devices per user.

Note: file_id in the Chunk and Permissions tables references the Metadata DB's files table, but as each service has its own database, this is an application-level reference, not a database foreign key.

6. Technology Stack

Layer	Technology
Language	Python 3.10
Metadata & Users DB	SQLite (simulation) -> PostgreSQL (production)
Chunk lookup DB	SQLite (simulation) -> DynamoDB / Cassandra (production)
Object storage	Local folders simulating nodes -> Amazon S3 (production)
Hashing	SHA-256 (hashlib)

Caching	Redis (production) - chunk lookups, sessions
Notifications	In-memory queue (simulation) -> Kafka/RabbitMQ + Redis (production)
Dev environment	Google Colab / Jupyter Notebook
Version control	Git & GitHub

7. Implementation Details

The simulation implements one Python class per service, each with its own SQLite database. A CloudStore class acts as the API Gateway, exposing `upload_file`, `download_file`, `update_file`, and `share_file`.

7.1 Chunking & Hashing

```
CHUNK_SIZE = 1024 * 1024 # 1 MB
```

```
def split_into_chunks(self, data):
    return [data[i:i+CHUNK_SIZE] for i in range(0, len(data), CHUNK_SIZE)]
```

```
def _hash_chunk(data):
    return hashlib.sha256(data).hexdigest()
```

7.2 Deduplication

Before storing a chunk, the Chunk Service checks if a chunk with the same hash already exists. If so, it is reused instead of being written again - this is what lets a new file version reuse unchanged chunks from a previous version.

7.3 Replication & Fault Tolerance

```
REPLICATION_FACTOR = 3
```

```
def _pick_nodes(self, chunk_hash, count):
    start = int(chunk_hash, 16) % self.num_nodes
    return [(start + i) % self.num_nodes for i in range(count)]
```

Each chunk is written to 3 nodes. On read, the first available replica is used - verified by marking 2 of 5 nodes 'down' and confirming the file still downloads correctly.

7.4 Access Control

Every download/update first checks the permissions table. A read-only user attempting an update receives a `PermissionError`.

7.5 Notifications & Offline Sync

On file upload/update, online devices are notified immediately; offline devices get the event added to a backlog, delivered on reconnect.

7.6 Quota Enforcement

Before any upload/update, the Users & Devices service checks if the operation would exceed the user's quota_bytes, and rejects it if so.

8. Screenshots

Screenshots of the simulation running in Google Colab, covering each scenario from Section 7.

```
import os
import sqlite3
import hashlib
import shutil
import uuid
import time

# Chunk size: 1 MB chunks (real systems use 4-8MB; small here for demo)
CHUNK_SIZE = 1024 * 1024

# Each chunk is stored on 3 different nodes for durability/fault tolerance
REPLICATION_FACTOR = 3

# Simulates the "Cloud blob store" (S3-equivalent)
STORAGE_ROOT = "cloudstore_data"

# Simulates a distributed storage cluster
NUM_STORAGE_NODES = 5

# Separate databases per service - mirrors "database separation per service"
METADATA_DB = "metadata_service.db"
USERS_DEVICES_DB = "users_devices.db"
CHUNK_DB = "chunk_service.db"

print("Configuration loaded.")

... Configuration loaded.
```

Fig 2: Setup (Task 0) - users and devices initialized

```
class StorageCluster:
    def __init__(self, root=STORAGE_ROOT, num_nodes=NUM_STORAGE_NODES):
        self.root = root
        self.num_nodes = num_nodes
        self.node_dirs = []
        for i in range(num_nodes):
            node_path = os.path.join(self.root, f"node_{i}")
            os.makedirs(node_path, exist_ok=True)
            self.node_dirs.append(node_path)
        self.failed_nodes = set()

    def _pick_nodes(self, chunk_hash, count):
        """Consistent hashing style: deterministic but spread-out node placement."""
        start = int(chunk_hash, 16) % self.num_nodes
        return [(start + i) % self.num_nodes for i in range(count)]

    def store_chunk(self, chunk_hash, data):
        nodes = self._pick_nodes(chunk_hash, REPLICATION_FACTOR)
        written_to = 0
        for node_id in nodes:
            if node_id in self.failed_nodes:
                continue
            path = os.path.join(self.node_dirs[node_id], chunk_hash)
            with open(path, "wb") as f:
                f.write(data)
            written_to.append(node_id)
        if not written_to:
            raise RuntimeError(f"Chunk {chunk_hash!s} could not be stored - all target nodes are down!")
        return written_to

    def read_chunk(self, chunk_hash):
        nodes = self._pick_nodes(chunk_hash, REPLICATION_FACTOR)
        for node_id in nodes:
            if node_id in self.failed_nodes:
                continue
            path = os.path.join(self.node_dirs[node_id], chunk_hash)
            if os.path.exists(path):
                with open(path, "rb") as f:
                    return f.read()
        raise FileNotFoundError(f"Chunk {chunk_hash!s} unavailable - all its replicas are missing or on failed nodes.")

    def simulate_node_failure(self, node_id):
        print(f"[FAULT TOLERANCE] Storage node {node_id} has gone DOWN.")
        self.failed_nodes.add(node_id)

    def recover_node(self, node_id):
        print(f"[FAULT TOLERANCE] Storage node {node_id} is back UP.")
        self.failed_nodes.discard(node_id)

    print("StorageCluster defined.")

... StorageCluster defined.
```

Fig 3: Chunked upload with replication across storage nodes

Task 2: Offline Device Reconnects (Notification Backlog)

Alice's laptop comes back online and receives the notification that was queued in the backlog while it was offline.

```
backlog_events = cs.notifications.reconnect_device(alice_laptop)
print("\nBacklog events delivered:", backlog_events)

... [NOTIFY] Device c85aa69d reconnected - delivering 1 backlog event(s)

Backlog events delivered: [{'file_id': '0db64389-55c7-4b9e-95ba-38bb5eee2196', 'file_name': 'trip_video.mp4', 'event': 'file_uploaded', 'timestamp': 1781421089.0134835}]
```

Fig 4: Offline device receives backlog notification on reconnect

Task 4: Chunked Download & Integrity Check (Q3 - File Retrieval)

Bob downloads the shared file. The Chunk Service reassembles it from chunks retrieved across storage nodes, and we verify the result matches the original byte-for-byte.

```
downloaded = cs.download_file("bob", file_id)
print(f"\nIntegrity check: downloaded == original -> {downloaded == fake_video}")

... [DOWNLOAD] Retrieved 'trip_video.mp4' v1 (2621440 bytes)

Integrity check: downloaded == original -> True
```

Fig 5: Chunked download and integrity check

```
updated_video = fake_video[:CHUNK_SIZE] + os.urandom(CHUNK_SIZE) # 1st chunk reused -> dedup
new_version = cs.update_file("alice", file_id, updated_video,
                             target_devices={alice_phone: True, alice_laptop: True})
print(f"\nNew version number: {new_version}")

... Chunk 0 (hash 9403b7cd...) -> deduplicated (already exists)
    Chunk 1 (hash 75668992...) -> stored on nodes [0, 1, 2]
[METADATA] File 0db64389 updated to version 2
[NOTIFY] Device 927b2ec9 -> 'file_updated' for trip_video.mp4
[NOTIFY] Device c85aa69d -> 'file_updated' for trip_video.mp4

New version number: 2
```

Fig 6: New version upload with chunk deduplication


```

print("Alice's files (Metadata Service):")
for row in cs.metadata.list_files("alice"):
    print(" ", row)

print("\nChunk records (Chunk Service):")
cur = cs.chunks.conn.cursor()
cur.execute("SELECT file_id, version, chunk_index, chunk_hash, chunk_size FROM file_chunks ORDER BY version, chunk_index")
for row in cur.fetchall():
    print(" ", row)

print("\nStorage node contents (Block Service):")
for node_dir in cs.storage.node_dirs:
    files = os.listdir(node_dir)
    print(f" {node_dir}: {len(files)} chunk(s) -> {files}")

... Alice's files (Metadata Service):
('b716df60-c716-4555-9aac-8ea298c361f1', 'trip_video.mp4', 2621440, 1, 'owner')

Chunk records (Chunk Service):
('b716df60-c716-4555-9aac-8ea298c361f1', 1, 0, 'd88aa5c197ff025b1b7ff50dfce228f5f26bb8733c683fd1e99544f3b01dbaf4', 1048576)
('b716df60-c716-4555-9aac-8ea298c361f1', 1, 1, '53df8bb8e33569a696b0037186c581d6cc2f4f3be724705def15cb794ae6436d', 1048576)
('b716df60-c716-4555-9aac-8ea298c361f1', 1, 2, 'bf2e716fd3588b65c0c9d9d13ef99547dc2a6d61e284b96e3eb496854b4f17b6', 524288)

Storage node contents (Block Service):
cloudstore_data/node_0: 2 chunk(s) -> ['bf2e716fd3588b65c0c9d9d13ef99547dc2a6d61e284b96e3eb496854b4f17b6', 'd88aa5c197ff025b1b7ff50dfce228f5f26bb8733c683fd1e99544f3b01dbaf4']
cloudstore_data/node_1: 2 chunk(s) -> ['bf2e716fd3588b65c0c9d9d13ef99547dc2a6d61e284b96e3eb496854b4f17b6', '53df8bb8e33569a696b0037186c581d6cc2f4f3be724705def15cb794ae6436d']
cloudstore_data/node_2: 2 chunk(s) -> ['bf2e716fd3588b65c0c9d9d13ef99547dc2a6d61e284b96e3eb496854b4f17b6', '53df8bb8e33569a696b0037186c581d6cc2f4f3be724705def15cb794ae6436d']
cloudstore_data/node_3: 2 chunk(s) -> ['d88aa5c197ff025b1b7ff50dfce228f5f26bb8733c683fd1e99544f3b01dbaf4', '53df8bb8e33569a696b0037186c581d6cc2f4f3be724705def15cb794ae6436d']
cloudstore_data/node_4: 1 chunk(s) -> ['d88aa5c197ff025b1b7ff50dfce228f5f26bb8733c683fd1e99544f3b01dbaf4']

```

Fig 10: Database and storage node inspection output

9. Future Scope

- Replace SQLite with PostgreSQL (metadata, users) and DynamoDB/Cassandra (chunk lookup) for production scale.
- Replace local-folder storage with Amazon S3 or a self-hosted MinIO cluster.
- Convert service classes into independently deployable microservices over REST/gRPC.
- Add a Redis caching layer for metadata and chunk-hash lookups.
- Use Kafka/RabbitMQ for notifications, with Redis for the offline backlog.
- Add end-to-end encryption for files at rest and in transit.
- Implement consistent hashing with virtual nodes for smoother scaling of storage nodes.
- Build a minimal web/desktop client demonstrating real-time sync via the API.